

Move Semantics

Move Semantics is a set of features (introduced in C++11) that allows the "ownership" of resources to be transferred from one object to another instead of making a full copy.

The Problem it Solves (The Waste of Deep Copies)

Before C++11, every time an object was **passed by value** or **returned** from a function, the **Copy Constructor** was called.

Imagine you have a Vector class managing an array of **100,000** integers on the Heap. While any kind of temporary object creation, like **returning/passing** to a function, the old C++ would:

1. Allocate a brand-new block of **100,000** integers on the Heap.
2. Copy all **100,000** integers over one by one (Deep Copy).
3. Immediately destroy the temporary original object and its Heap memory.

This is incredibly inefficient. You are doing massive amounts of work to copy data.

1. The Foundation: L-values vs. R-values

Every expression in C++ belongs to a "**value category**." The two most important are:

L-values (Locator Values): These are objects that occupy an identifiable, permanent location in memory. They have a name, and they live beyond a single line of code.

- **Rule of thumb:** If you can safely take its memory address using **&**, it is an **l-value**.
- **Examples:** Variables like **int x**, **Dog myDog**, or **string s**.

R-values (Read Values): These are temporary. They do not have a permanent memory address. They exist purely to be evaluated on a single line of code, and then they instantly vanish.

- **Rule of thumb:** If it doesn't have a **name** and is about to die, it is an **r-value**.
- **Examples:** Literal numbers like **5**, the result of math like **x + y**, or an **unnamed** temporary object returned from a function.

```
int x = 5;
// 'x' is an l-value (it has a memory location and a name).
// '5' is an r-value (it is just a temporary concept of the number 5).

x = 10; // VALID: You can assign a new value to an l-value.
10 = x; // ERROR: You cannot assign something to an r-value.
```

The compiler automatically looks at the object you are passing. If it is an "**l-value**", it calls the safe, slow copy version. If it is an "**r-value**", it automatically routes it to the fast, move version.

3. The std::move() Deception

std::move() does exactly one thing: **It is used for type-casting**. It forcibly converts an “l-value” into an “r-value” reference (&&).

Why do we need this? Sometimes we have a named variable (an l-value) that we are completely done using, but the compiler doesn't know that. If we pass it to another function, it will do a deep-copy it. By using **std::move()**, we are explicitly telling the compiler: "I know this object has a name, but I am done with it. Now treat it like a temporary r-value so the next function can steal its guts."

```
string name1 = "Alice is a very long string stored on the heap";

// The compiler sees 'name1' is an l-value. It plays it safe and does a deep copy.
string name2 = name1;

// The compiler sees std::move(). It treats 'name1' as an r-value.
// It calls the Move Constructor. 'name3' steals the memory.
string name3 = std::move(name1);

// DANGER: 'name1' is now an empty shell! Its pointers were stolen.
cout << name1; // Outputs nothing.
```

Move Constructor

This mechanism single-handedly made C++ dramatically faster by eliminating unnecessary deep copies.

The Solution: Move Constructor (Stealing)

It says: "If the original object is about to be destroyed anyway, don't copy its data. Just **steal** its pointer."

The Analogy: Imagine your friend has a car (the Heap memory) and the keys to the car (the Stack pointer).

- A **Copy Constructor** forces you to go to a factory, build an exact replica of the car, and get your own keys.
- A **Move Constructor** just takes your friend's keys and leaves them with nothing. You now own the car. It is instantaneous.

The Syntax and Signature

To tell the compiler you want to move instead of copy, you use an **r-value reference**, denoted by a **&&**.

```
// Move Constructor Signature
ClassName(ClassName &&other) noexcept;
```

Note: Move operations should always be marked **noexcept** so standard libraries (like std::vector) can safely use them during resizing.

How to write a Move Constructor

Here are the two critical steps:

1. **Steal the data:** Point your new object's pointers to the old object's Heap memory.
2. **Nullify the source:** You **MUST** set the old object's pointers to **nullptr**. If you forget this, when the temporary object dies a microsecond later, its destructor will **delete** the memory you just stole

When is it called automatically?

While you can force a move using **std::move()**, the compiler is incredibly smart. It will automatically call your Move Constructor whenever it knows an object is temporary.

Move Assignment Operator

It allows an **already existing** object to "steal" resources from a temporary object during an assignment (=).

1. The Syntax

```
ClassName &operator=(ClassName &&other) noexcept;
```

2. The Implementation Steps

1. **Self-Assignment Check:** Ensure you aren't moving an object into itself (if (this == &other)).
2. **Deallocate Current Resource:** Free the memory the object currently holds.
3. **Transfer Ownership:** Copy the pointer from the other object to this object.
4. **Nullify the Source:** Set the **other object's** pointer to **nullptr** so its destructor doesn't delete the memory you just stole.
5. **Return** *this.

```
class Student {
private:
    char *name;
    int regno;

public:
    // 1. Standard Constructor
    Student(const char *n, int r) : regno(r) {
        name = new char[strlen(n) + 1];
        strcpy(name, n);
    }

    // 2. Destructor
    ~Student() { delete[] name; }
```

```

// 3. Move Constructor
Student(Student &&other) noexcept {

    this->name = other.name; // Transfer resources
    this->regno = other.regno;

    other.name = nullptr; // Nullify source to prevent double-deletion
    other.regno = 0;
}

// 4. Move Assignment Operator
Student &operator=(Student &&other) noexcept {

    if (this == &other) { return *this; } // Self Assignment Check

    delete[] name; // Free existing memory

    this->name = other.name; // Transfer resources
    this->regno = other.regno;

    other.name = nullptr; // Nullify source
    other.regno = 0;

    return *this;
}

};

int main() {

    Student s1("Ismail", 1);

    // Move Constructor
    Student s2(std::move(s1)); // s1 becomes 'empty', s2 becomes 'Ismail'

    Student s3("Temporary", 0);

    // Move Assignment Operator
    s3 = std::move(s2); // s2 becomes 'empty', s3 becomes 'Ismail'

    return 0;
}

```